

WAS: Wear Aware Superblock Management for Prolonging SSD Lifetime

Shunzhuo Wang[†], Fei Wu[†], Chengmo Yang[‡], Jiaona Zhou[†],
Changsheng Xie[†] and Jiguang Wan[†]

[†]Huazhong University of Science and Technology, China

[‡] University of Delaware, USA

Corresponding author: Fei Wu, wufei@hust.edu.cn

{wangshunzhuo, zhoujiaona, cs_xie, jgwan}@hust.edu.cn, chengmo@udel.edu

ABSTRACT

Superblocks are widely employed in SSDs for improving performance. However, the standard superblock organization which links blocks with the same block ID across planes into one superblock leads to SSDs' ineluctable lifetime waste due to inter-block wear tolerance variations. This work proposes a wear-aware superblock management, called WAS, which (1) dynamically organizes superblocks according to real-time block wear levels to make strong blocks relieve wear on weak ones, and (2) employs a wear-based garbage collection scheme to reduce inter-block wear gap. Comprehensive experiments are carried out in SSDsim. Results show that WAS greatly prolongs SSD lifetime by 51.3% compared with the state-of-the-art superblock management.

KEYWORDS

SSDs, Superblock, Wear Tolerance Variation, Wear-based Garbage Collection, Lifetime

1 INTRODUCTION

NAND flash-based SSDs [1] have been widely employed in many embedded and computer systems because of their high performance, non-volatility, low power consumption, and shock resistance [10]. To make maximum use of the multi-level internal SSD parallelism [9], SSD manufactures usually aggregate blocks with the same block ID across multiple planes into one superblock to deliver the greatest possible throughput and promote system performance [12]. The controller regards superblocks as the granularity of block management to facilitate easy file management and memory maintenance. Thus, all blocks within the same superblock have to undergo the same Program/Erase (P/E) cycle. However, equalizing the P/E cycles across all blocks within one superblock overlooks process variation, which reduces the total P/E cycles and shortens SSD lifetime.

Due to the manufacturing process variation [6], flash cells reveal unbalanced bit error characteristics [7], resulting in unbalanced wear tolerance across different blocks. That is,

the wear of flash blocks which have experienced the same P/E cycles is usually uneven and different blocks are usually armed with significantly different endurance, as shown in Figure 1(a). Likewise, there is inter-page wear tolerance variation within each block and bit error rates (BERs) of pages increase at highly different speed. Moreover, the weakest page reveals the highest BER during the whole block lifetime, as shown in Figure 1(b).

Unfortunately, the standard organization of superblocks which overlooks the actual wear tolerance of blocks causes ineluctable lifetime waste. On the one hand, weak blocks with low wear tolerance have to suffer from the same P/E cycles as the others within the same superblock. Consequently, these weak blocks break down earlier and turn to bad blocks. When bad blocks are too many to deliver the expected performance of the SSD, the SSD is regarded as "fail", while most blocks are still under-utilized at that moment. On the other hand, the shift of block management granularity from blocks to superblocks weakens the efficiency of data management schemes (i.e., Garbage Collection and Wear-Leveling). Due to inter-block wear tolerance variation, it's difficult to find an accurate wear indicator for superblocks. As a result, existing wear-leveling schemes [2, 4, 14, 17, 20] can't efficiently evenly wear out superblocks. Regarding garbage collection, a larger reclaiming granularity leads to higher data migration overhead.

This work proposes a novel wear aware superblock management, called WAS, which efficiently reduces wear gap across blocks and prolongs the lifetime of SSDs. To this end, a fast and accurate block wear detection scheme is designed, which leverages inter-page wear tolerance variation within blocks and efficiently detects the block wear by only measuring the BER of the weakest page. To relieve the block wear gap, WAS dynamically organizes superblocks according to the real-time wear information of blocks and links free strong blocks rather than those with the fixed block ID across planes into one superblock. The flexible superblock management makes strong blocks relieve weak blocks and prevents the SSD from suffering premature death. Note that WAS can function well as long as all the planes are armed with the same amount of free blocks. WAS not only keeps the advantages of high throughput but also eliminates potential efficiency drop in data management schemes. Furthermore, WAS employs a wear-based garbage collection scheme to level wear of blocks within planes. Both reclaiming cost and the block wear are taken into account in selecting victim blocks. The block with the most invalid pages from the strong garbage blocks (blocks to be recycled) is selected as the victim block. To avoid increasing wear gap caused by cold data reside in strong

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '19, Las Vegas, NV, USA

© 2019 ACM. 978-1-4503-6725-7/19/06...\$15.00

DOI: 10.1145/3316781.3317929

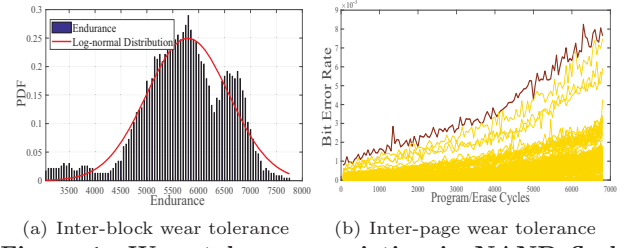
blocks for a long period, WAS gives priority to reclaiming them to further relieve wear unevenness within planes. By harmonizing the wear aware superblock organization and the wear-based garbage collection, WAS efficiently evens out wear levels across blocks and prolongs the lifetime of SSDs. A series of comprehensive experiments based on SSDsim [9] are conducted with 4 real-world traces [13]. Experiment results show that the lifetime of SSDs could be significantly promoted with negligible storage overhead. To be specific, the lifetime could be improved by 51.3% (/106.6%) compared with the traditional superblock management with (/without) the state-of-the-art wear-leveling scheme.

The rest of this paper is organized as follows: Section 2 presents the basics of SSDs and summarizes related works. Motivation is shown in section 3. The proposed wear aware superblock management is shown in section 4. Section 5 presents experiments and analyzes the results. Finally, the conclusion is given in section 6.

2 BACKGROUND

As an alternative to HDDs, NAND flash-based SSDs are widely used in the storage market [10]. SSDs are sophisticated electronics and armed with multi-level parallelism space[9]. SSDs usually possess more channels and each of which consists of multiple chips. Each chip is composed of multiple dies and each die is composed of multiple planes. Each plane includes hundreds of blocks, and is equipped with an independent data register to increase SSD throughput. To make full use of the multi-level parallelism and deliver the greatest possible throughput, SSD manufacturers usually aggregate blocks with the same block ID across multiple planes into one superblock. This improves performance by increasing the visible size of the NAND flash data register and the visible size of individual blocks. Instead of blocks, superblocks are regarded as the granularity of block management to facilitate easy file and memory management [12, 18]. To function well during the whole lifetime, all blocks with the same block ID must be in the same states (free or not). It necessitates the data management schemes in superblock level.

Due to out-of-place update, data can be directly updated to their residing pages unless their belonging blocks are erased. Garbage collection is employed in SSDs to reclaim stale data [5, 8, 15]. These schemes concentrate on proposing better metrics in victim block selection to improve the efficacy of garbage collection. Since a flash block can only endure limited P/E cycles before it becomes worn-out and corrupted, wear-leveling schemes [4, 14, 20] are designed to even out wear across blocks to expand the lifetime of SSDs. With an indepth understanding of the structure of the flash cell, researchers continue to explore different ways for measuring the block wear. Considering the fact that a flash cell wears only in the process of programming and erasing, and the influence of some error inducements (such as read disturb, retention and program interference) on BERs is reversible, Yang et al. [20] discarded P/E cycles and BERs and selected the write errors measured via “read-after-write validation” as the metric of the block wear. In this approach, data are read right after they are programmed and the BER of data is detected as the write errors of the page. The maximal write errors among all pages in one block are considered as the flash block wear. The scheme efficiently takes into account the influence of reversible error inducements and accurately measures the block wear.



(a) Inter-block wear tolerance (b) Inter-page wear tolerance
Figure 1: Wear tolerance variation in NAND flash chip. (a) shows the endurance per block based on measurements across all blocks. The endurance shows significant difference across blocks within the same chip. (b) illustrates BERs of all pages within the same block. The BERs grow at highly different speed among pages and the weakest page reveals the highest BER all over the block’s lifetime.

However, it takes an extra reading after every programming to detect page wear, resulting in high performance penalty.

3 MOTIVATION

3.1 Unbalanced Wear Tolerance

Flash blocks are made up of millions of flash cells, which reveal unbalanced bit error characteristics due to the manufacturing process variation [6]. Moreover, the cell-level error characteristics propagate up to pages and blocks. To study page-level/block-level error characteristics, this work explores the inter-page/inter-block wear tolerance based on a real flash chip [19], its characteristics are shown in Table 1.

Table 1: NAND FLASH CHARACTERISTICS

page size	16KB	block size	9MB (576 pages)
read time	75μs	program time	1100 μs
erase time	10ms	ECC	120 bits per 1 KB

Block-level unbalanced wear tolerance. This work models the impact of process variation on block endurance according to test results. As shown in Figure 1(a), there is a significant endurance difference across blocks. The minimum block endurance en_{min} is 3383 and the maximum block endurance en_{max} is 7700. This work approximately models the block endurance as a bounded Gaussian distribution, shown below:

$$f(en) = \begin{cases} \frac{1}{\sqrt{2\pi}\delta} e^{-\frac{(en-\mu)^2}{2\delta^2}} & en_{min} \leq en \leq en_{max} \\ 0 & \text{others} \end{cases} \quad (1)$$

The standard deviation δ is 826.9 and the average μ is 5578.

Page-level unbalanced wear tolerance. This work explores the wear trend of all pages of one block during the whole SSD lifetime. Interestingly, there is unbalanced wear tolerance among pages, and different pages of one block deteriorate at different rates, as shown in Figure 1(b). Specifically, weak pages reveal high BERs all over the lifetime of the block. What’s more, the BER of the weakest page has consistently been the highest during the whole endurance of the block. The page-level unbalanced wear tolerance is essentially from the flash cell process variation [6], and the pages within one block suffer from the similar degree of wear due to data randomization technology [3] during usage. Thus, the relative wear difference trend across pages is steady. Accordingly, the

BER of the weakest page can be used as the metric of block wear throughout the lifetime of that block.

3.2 Ineluctable Lifetime Waste of SSDs

SSD manufactures usually coordinate blocks with the same block ID across planes into one superblock for improving performance. However, the standard superblock organization causes ineluctable lifetime waste of SSDs.

Untimely break-down of weak blocks. The endurance of weak blocks is much smaller than the strong ones, but they have to endure the same P/E cycles as the strong block within the same superblock. As shown in Figure 2, all blocks in superblock b have to suffer from the same level of wear without distinction. Consequently, weak blocks early approach their endurance limits and break down, which in turn exacerbates the growth of bad blocks. When bad blocks are too many to tolerate, the SSD is regarded as “fail”. Regarding the strong blocks, they could have endured more P/E cycles but they are under-utilized. The untimely break-down of weak blocks accelerates the SSD failure and causes ineluctable waste of SSD total P/E cycles.

High superblock-level garbage collection overhead. In the standard superblock organization, garbage collection must be carried out at superblock granularity. Although the victim superblock is selected in a greedy fashion according to reclaiming cost, some blocks where most data are valid may end up being recycled because other blocks within the same superblock have a mass of stale data, which brings high migration overhead and has adverse effect on the SSD lifetime. As shown in Figure 2, although the second block (the block with red frame) in superblock h has few invalid data, it is still reclaimed together with other blocks within superblock h .

Less measurable wear metric for superblocks. Because there is unbalanced wear tolerance cross blocks, blocks which have experienced the same P/E cycles reveal significantly different wear levels. It is difficult to find an appropriate

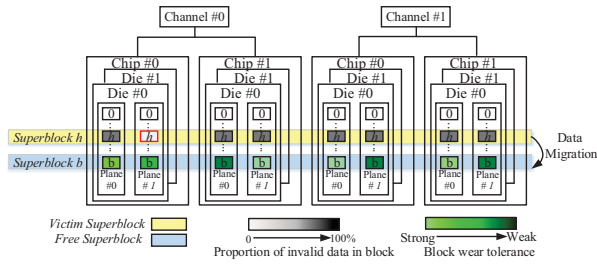


Figure 2: Superblock block diagram. Blocks with significantly different wear tolerance have to suffer from the same level of wear. Blocks with few invalid data may be reclaimed because others within the same superblock have a mass of invalid data.

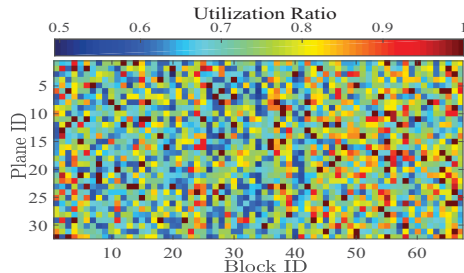


Figure 3: Under-utilized blocks when the SSD fails.

wear metric to measure the actual wear of a superblock. Thus, existing wear-leveling schemes can't efficiently even out wear gap across superblocks to increase SSD utilization.

To clearly reveal this issue, a simulation with the real-world trace ts0 [13] is carried out. A simulated SSD is generated according to the block endurance model as shown in Formula (1). Experiments based on the SSDsim [9] illustrate that the SSD utilization is only 72.08% and that of most blocks are less than 80%, some blocks are utilized as low as 50% when the SSD fails, as shown in Figure 3.

Motivated by those three observations, this work aims to propose a dynamic superblock management to make strong blocks relieve the stress on weak blocks, mitigate high migration overhead, and even out the wear gap across blocks.

4 DESIGN

4.1 Overview

To mitigate the lifetime waste of SSDs in the standard superblock organization, a wear aware superblock management, called WAS, is proposed in this work, which (1) dynamically organizes superblocks according to the real-time block wear to make strong blocks relieve wear on weak ones and (2) employs a wear-based garbage collection scheme to reduce wear gap across blocks. As shown in Figure 4, there are three function modules and one memory module in WAS: *Wear Detector*, *Superblock Organizer*, *Wear-based Garbage Collector*, and *Block Wear Table*. The wear detector module leverages the inter-page wear tolerance variation to efficiently detect the block wear, then it is stored in the block wear table. The detected wear is a key metric for the superblock organizer module and the wear-based garbage collector. The superblock organizer dynamically adjusts superblock organization according to the real-time block wear and coordinates strong blocks instead of those with the same block ID across planes into superblocks. The wear-based garbage collector aims to even out the wear across blocks within the same plane. It takes both the block wear and reclaiming cost into account in selecting victim blocks to relieve the intra-plane wear gap.

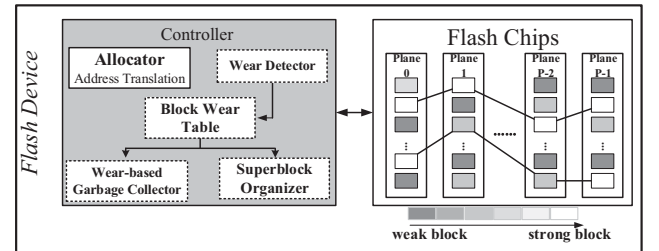


Figure 4: The Architecture of WAS.

4.2 Wear Detector and Block Wear Table

To relieve the high performance penalty associated with “read-after-write validation” in the block wear detection [20], the wear detector in WAS utilizes the inter-page wear variation to measure the block wear at the cost of negligible performance overhead. As mentioned in section 3.1, the relative trend of wear difference among pages within one block is steady. The weakest page consistently displays the maximal wear in the residing block during the whole lifetime. Because one block is considered to be unreliable only if the BER of any page in the block exceeds the capability of ECC, the block wear depends on the weakest page. Therefore, the block wear

detector measures ONLY the weakest page instead of all pages in one block to efficiently determine the block wear.

To serve the superblock organization and wear-based garbage collector, WAS establishes the block wear list to record states of all blocks. Considering that the blocks within one superblock are across planes, WAS manages block states at plane granularity. Each node in the block wear table contains the following information:

- *Block ID* indexes the block in certain plane;
- *Page ID* indexes the weakest page and it is set before leaving the factory.
- *Block wear* stores the error bits of the weakest page and indicates the real-time block wear;
- *Erase time* indicates the most recent-erasing time of the block.

Furthermore, a *erase counter* is employed and stored in RAM to indicate the global erase time. Once any block is erased, the erase counter increases by one and the erase time of the block node is set at the current value of the erase counter. The value in the erase counter steadily increases during the whole lifetime of SSDs, thus WAS utilizes it as a time indicator.

RAM overhead for the block wear table is negligible. For example, regarding a 1TB SSD with 20% over-provisioning space where the block size is 9MB, it includes $\frac{1TB \times 1.2}{9MB} \approx 1398100$ blocks. 2B is enough to index all blocks within one plane and 2B is enough for Page ID. For given ECC of 120bits per 1KB, 2B is enough for the block wear and this work uses 5B to indicate the maximum erase count of the SSD. It takes only 11B to indicate all information of one block state node and RAM overhead for the block wear table is $1398100 \times 11B \approx 14.67MB$.

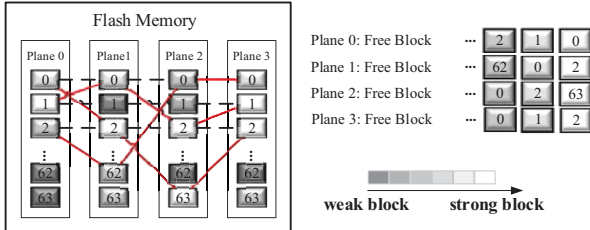


Figure 5: Wear Aware Superblock Organization. The solid lines depict the wear aware superblock organization and the dotted lines presents the standard superblock organization.

4.3 Wear Aware Superblock Organization

To eliminate the untimely break-down of weak blocks in the standard superblock organization, WAS dynamically organizes superblocks according to real-time wear and coordinates strong blocks across planes into superblocks. When one free superblock is needed, WAS traverses free blocks of planes for superblock organization. Instead of blocks with the same block ID, WAS selects the strongest free block with the lowest wear level from each plane to constitute one superblock. As shown in Figure 5, the superblock is organized in a “polyline” fashion (with blocks with the lowest wear) rather than in a “horizontal” fashion (with blocks with the same block ID).

The wear aware superblock organization prevents blocks with different wear tolerance from suffering the same degree

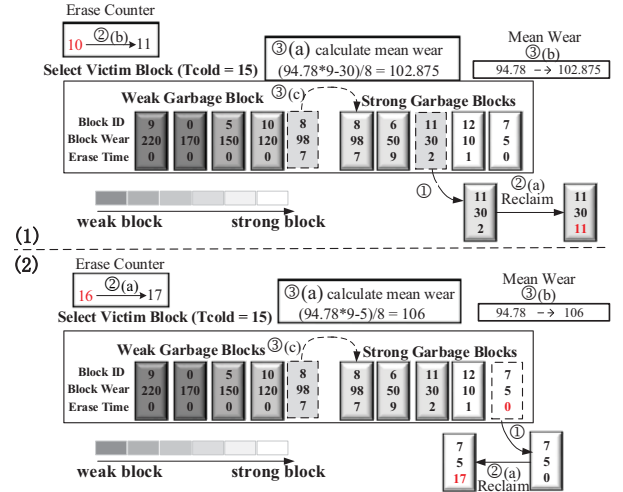


Figure 6: Wear Based Garbage Collection.

of damage and prevents weak blocks from premature death. In wear aware superblock organization, strong blocks have given higher priority to make up superblocks and endure the wear that was supposed to damage weak blocks within the same plane. It makes strong blocks to relieve weak blocks and evens out the wear difference across blocks within the same plane. What's more, the dynamic organization manages to break through the constraint that the block management has to be carried out at superblock granularity because wear aware superblock organization can function well as long as there is the same amount of free blocks in each plane. This breakthrough is helpful to reduce data migration overhead and find a way to relieve wear gap across blocks.

4.4 Wear-Based Garbage Collection

Thanks to wear aware superblock organization, the victim blocks are not selected at superblock granularity but from multiple planes at block granularity. However, the existing garbage collection schemes may end up enlarging the wear gap across blocks due to asymmetric write-hotness [11]. Regarding weak blocks storing hot data, they are recycled according to the reclaiming cost (the amount of invalid data in blocks). On the contrary, some strong blocks retaining cold data can be seldom reclaimed and endures little wear. This worsens the wear gap across blocks. Thus, this work designs wear-based garbage collection to reach a tradeoff between reclaiming efficacy and wear leveling.

WAS adopts a wear-based garbage collection, which takes both reclaiming cost and wear are taken into account in victim block selection. Specifically, WAS selects victim blocks only in the strong garbage blocks in each plane. WAS traverses the strong blocks in each plane and selects the block with the most invalid data as the victim block for high reclaiming cost. Once some blocks which store cold data for a long period are found, WAS gives high priority to reclaiming them to even out the block wear.

All garbage blocks are divided into two categories according to the block wear: strong garbage blocks and weak garbage blocks. The mean wear of all garbage blocks in the plane is maintained for identifying strong or weak garbage blocks. If the wear of a garbage block exceeds the mean wear, this block is regarded as a “weak” garbage block. Otherwise, the

block is considered “strong”. A threshold T_{cold} is set to detect whether there exists cold data retaining in blocks for a long period. If the difference between the current value of the global *erase counter* and the erase time of a garbage block is more than T_{cold} , data in this block are considered as cold data.

If there is no block storing cold data, WAS selects the victim block from the strong garbage blocks in a greedy fashion according to reclaiming cost, as shown in Figure 6(1). WAS traverses all strong blocks and selects block 11 armed with the most amount of invalid data as the victim block (Step ①). Then WAS erases the block after migrating valid data and modifies the block state information (Step ②). The erase counter adds 1 and the erase time of the reclaimed block state node is set at the current value of the erase counter. Lastly, WAS adjusts the garbage block list (Step ③). The mean wear is updated and WAS adjusts all garbage blocks. If there is a garbage block storing cold data for too long, WAS reclaims the block with a high priority, as shown in Figure 6(2). Because block 7 stores cold data, the block is never erased before. But with the proposed scheme, once the difference between erase time of the block and erase counter exceeds the T_{cold} , the block is selected as the victim block regardless of its amount of invalid data. The corresponding block state lists are maintained after block 7 is reclaimed.

5 EVALUATION

5.1 Experiment Setup

The proposed WAS is evaluated based on SSDsim [9] with the standard superblock management with and without the ERA wear leveling scheme [20] where the maximal block wear within one superblock is used as the wear indicator of the superblock. In experiments, a virtual 16GB SSD with 17.78% over-provisioning space is constructed and endurance of all blocks conforms to Formula (1). There are four channels in the simulated SSD and each channel consists of two chips. Each chip includes two dies and two planes constitute one die. Four real-world workloads [13] are chosen in experiments, with their characteristics presented in Table 2. This work sets the threshold of bad blocks (T_{bad}) at 5% [16] and SSDs are considered as “fail” once the ratio of bad blocks is more than T_{bad} . T_{cold} is set at 15 to determine whether the garbage blocks store the cold data. For the simulated 16GB SSD, RAM overhead for the block wear table is 23.03KB.

Table 2: Characteristics of 4 real-world workloads

	# of request	Write ratio	Avg size of write request	Avg size of read request
exchange	599999	68.0%	28.1 sectors	30.3 sectors
hm0	3993315	64.5%	16.6 sectors	14.7 sectors
src0	1557813	88.7%	16.3 sectors	14.2 sectors
ts0	1801487	82.4%	16.0 sectors	27.4 sectors

5.2 Lifetime and Performance

To clearly display the positive effect of proposed WAS on the lifetime of SSDs, this work comprehensively compares WAS with the superblock management without and with the ERA wear leveling scheme [20]. This work regards the endurable maximum amount of write requests before SSD failure as the lifetime indicator. WAS greatly promotes the lifetime of SSDs by 106.6% and 51.3% compared with the traditional superblock management without and with the ERA wear leveling scheme.

This work records states of all blocks during the whole lifetime and regards standard deviation of BERs detected via “read-after-write validation” of all blocks as the metric to indicate wear unevenness of SSDs. The larger the value is, the more uneven the wear of SSDs is. Figure 7(a) shows the wear unevenness under three different superblock management schemes. The standard superblock management worsens wear evenness of SSDs. The growth of wear unevenness slows down by employing the ERA wear leveling scheme, while WAS efficiently relieves wear gap across blocks and largely reduces wear unevenness at a very low level. WAS utilizes wear aware superblock management to reduce the wear gap across blocks with the same block ID and wear-based garbage collection to even out blocks in the same plane. Thus, wear unevenness of the SSD decreases at first. Since all planes have to endure the same level of wear, the wear gap across planes gradually increases due to inter-plane unbalanced wear tolerance, but the wear unevenness of SSDs is always at a low level. Wear unevenness leads to the early break-down of weak blocks and these blocks turn to bad blocks. Figure 7(b) shows the variation of bad blocks during the whole lifetime of SSDs. The ERA wear leveling scheme can delay the appearance of bad blocks, but it can’t avoid the early break-down of weak blocks. Owing to favorable wear evenness caused by WAS, all blocks are fully utilized and all bad blocks almost come out at the same time.

To further explore positive effects of WAS, block states are shown in Figure 7(c-d) when the SSD suffers from failure. Compared to the standard superblock management, WAS dynamically organizes superblocks according to the real-time block wear so that blocks endure wear stress corresponding to their wear tolerance. This efficiently relieves the wear gap caused by inter-block unbalanced wear tolerance. Figure 7(e) shows the block state under the circumstance of SSD failure. The SSD is fully utilized and its utilization ratio reaches up to 94.26%.

To understand how WAS functions on leveling wear of blocks, this work presents the wear variation of two blocks, namely the 6th block and the 19th block in the block set with the same block ID, under three different schemes mentioned above during the whole lifetime, as shown in Figure 7(f). The 6th block is a weak block with low wear tolerance, while the 19th block is strong. In the standard superblock management, two blocks have to endure the same level of wear and they wear at more-or-less the same speed during the whole lifetime. The ERA wear leveling scheme can slow down the growth of wear to some extent while it can’t preclude these two blocks with different wear tolerance from suffering the same level of wear. While WAS makes strong blocks relieve the wear of weak blocks by dynamically organizing superblocks. Consequently, the wear gap of the two blocks gets closer.

Figure 7(g-h) show the write and read performance under three superblock management schemes. The ERA wear leveling applies the “read-after-write validation” to detect the real-time wear of blocks, which leads to significant performance cost. Because the performance penalty of the block wear detection in WAS is negligible and WAS separately selects the victim block in each plane instead of the victim superblock in a greedy fashion to reduce the data migration overhead, WAS shows the similar performance with the standard superblock organization.

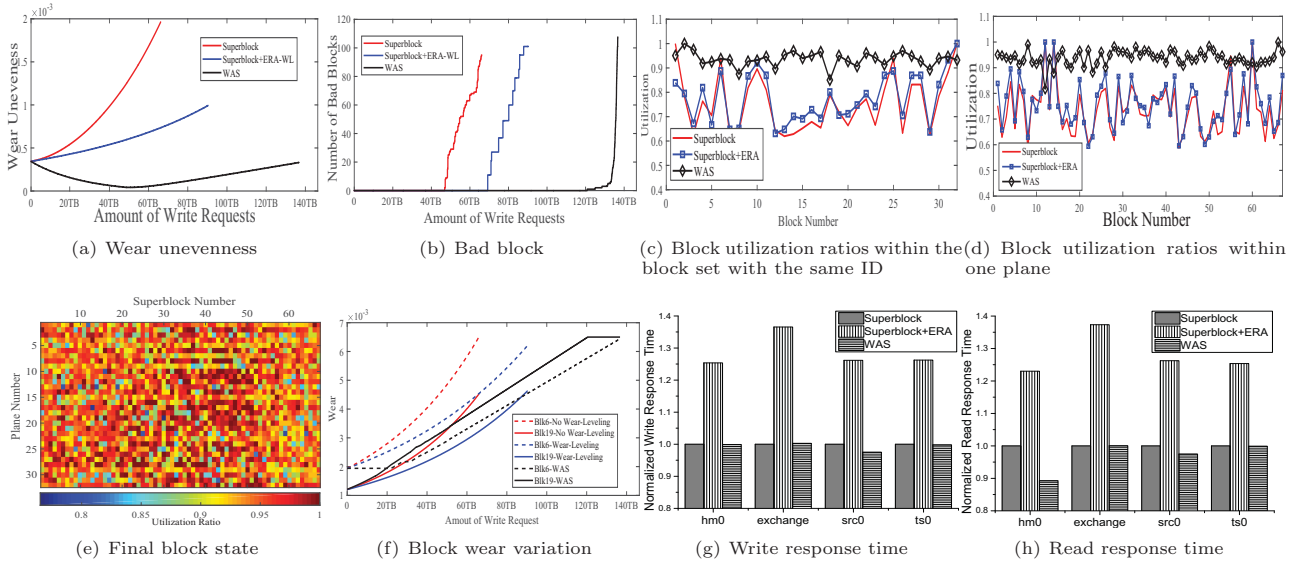


Figure 7: Simulation results. This work compares WAS with the standard superblock management with/without ERA wear leveling in lifetime and performance. (a) shows the wear effect of WAS; (b) illustrates the number variations of bad blocks during the whole lifetime; (c)-(e) show the utilization of blocks; (f) represents how WAS works; (g)-(h) show the performance influence of WAS.

6 CONCLUSION

Motivated by the fact that the standard superblock organization leads to the ineluctable lifetime waste of SSDs due to unbalanced wear tolerance across blocks, this work proposes a wear aware superblock management (WAS) to improve the lifetime of SSDs. WAS dynamically organizes superblocks according to the real-time block wear and coordinates blocks with low level of wear instead of blocks with the same block ID across planes into superblocks to make strong blocks relieve the stress on weak blocks. What's more, WAS utilizes wear-based garbage collection which takes wear into account in selecting victim blocks besides reclaiming cost to even out the wear difference across blocks. Experiment results demonstrate the effectiveness of the proposed design. Compared to the state-of-the-art superblock management, WAS prolongs the lifetime of SSDs by 51.3% at the cost of negligible performance impact and RAM space overhead.

ACKNOWLEDGMENTS

This work was supported by NSFC No. 61872413, No. U1709220, No. 61821003 and Wuhan Science and technology Project No. 2017010201010108, and Shenzhen basic research project No. JCYJ20170307160135308, JCYJ20170818162129916, and National Key Research and Development Program of China No.2018YFB10033005 and the Fundamental Research Funds for the Central Universities No.2016YXMS019, and the 111 Project (No. B07038). This work is also supported by Key Laboratory of Data Storage System, Ministry of Education.

REFERENCES

- [1] N. Agrawal, V. Prabhakaran, and et al. 2008. Design Tradeoffs for SSD Performance. *Proc. of ATC* (2008), 57–70.
- [2] A. Ban. 2004. Wear Leveling of Static Areas in Flash Memory. *US Patent* (2004), 6230233.
- [3] J. Cha and S. Kang. 2013. Data Randomization Scheme for Endurance Enhancement and Interference Mitigation of Multilevel Flash Memory Devices. *Proc. of ETRI* 35 (2013), 166–169.
- [4] Y. H. Chang, J. W. Hsieh, and T. W. Kuo. 2007. Endurance Enhancement of Flash-Memory Storage Systems: An Efficient Static Wear Wear-Leveling Design. *Proc. of DAC* (2007), 212–217.
- [5] M. Chiang and R. Chang. 1999. Cleaning Policies in Mobile Computers Using Flash Memory. *Proc. of Journal of Systems and Software* 48(3) (1999), 213–231.
- [6] M. Dietrich and J. Haase. 2012. Process Variations and Probabilistic Integrated Circuit Design. *Proc. of Springer* (2012).
- [7] G. Dong, N. Xie, and T. Zhang. 2012. Techniques for Embracing intra-cell Unbalanced Bit Error Characteristics in MLC NAND Flash Memory. *Proc. of GLOBECOM Workshops* (2012).
- [8] L. Han, Y. Ryu, and et al. 2006. CATA: A Garbage Collection Scheme for Flash Memory File Systems. *Proc. of Ubiquitous Intelligence and Computing* 4159 (2006), 103–112.
- [9] Y. Hu, H. Jang, and et al. 2012. Exploring and Exploiting the Multilevel Parallelism Inside SSDs for Improved Performance and Endurance. *Proc. of TOC* 62 (2012), 1141–1151.
- [10] S. Jeong, K. Lee, and et al. 2013. I/O Stack Optimization for Smartphones. *Proc. of ATC* (2013), 309–320.
- [11] Y. X. Luo, Y. Cai, and et al. 2015. WARM: Improving NAND Flash Memory Lifetime with Write-hotness Aware Retention Management. *Proc. of DATE* (2015), 11:1–11:14.
- [12] Micron. 2016. TN-29-28: Memory Management in NAND Flash Arrays Overview. (2016).
- [13] D. Narayanan, A. Donnelly, and A.I. Rowstron. 2008. Write off-loading: Practical power management for enterprise storage. *Proc. of FAST* (2008), 253–267.
- [14] Y. Pan, G. Dong, and T. Zhang. 2013. Error Rate-Based Wear-Leveling for NAND Flash Memory at Highly Scaled Technology Nodes. *Proc. of TVLSI* (2013), 1350–1354.
- [15] M. Rosenblum and J. K. Ousterhout. 1992. The Design and Implementation of a Log-Structured File System. *Proc. of TOCS* 10(1) (1992), 26–52.
- [16] B. Schroeder, R. Lagisetty, and A. Merchant. 2016. Flash Reliability in Production: the Expected and the Unexpected. *Proc. of FAST* (2016), 67–80.
- [17] X. Shi, F. Wu, and et al. 2018. Program Error Rate-based Wear Leveling for NAND Flash Memory. *Proc. DATE* (2018), 1241–1246.
- [18] Ying Y. Tai. 2016. High Performance FTL for PCIe/NVMe SSDs. *Proc. of Flash Memory Summit* (2016).
- [19] TOSHIBA. 2016. TOSHIBA 3D Flash Memory Toggle DDR2.0 Technical Data Sheet. (2016).
- [20] M. Yang, Y. Chang, and et al. 2013. New ERA: New Efficient Reliability-Aware Wear Leveling for Endurance Enhancement of Flash Storage Devices. *Proc. DAC* (2013), 163:1–163:6.